ANALISI CODICE

Analizzare il programma senza eseguirlo, individuare caratteristiche non standard o errori di sintassi/logica e proporre soluzione per ogni riga/blocco specificandolo.

CODICE DA ANALIZZARE OVERVIEW:

Il codice a prima vista sembra essere un rudimentale programma con opzioni di risposta limitate e con input riconoscitivi non espliciti, rendendo difficile per l'utente sapere quali siano le sue opzioni. La definizione della funzione è erroneamente messa DOPO una chiamata alla funzione in output, quindi in ordine inverso. Inoltre, l'input dell'utente non viene normalizzato e le condizioni di riconoscimento sono molto specifiche rendendo difficile soddisfare la richiesta desiderata. Ha inoltre alcuni errori di indentazione e sintassi. Vediamo ora in dettaglio il resto:

```
1   import datetime
2
3   while True
4       comando_utente = input("Cosa vuoi sapere? ")
5       if comando_utente == "esci":
6           print("Arrivederci!")
7       break
8       else:
9           print(assistente_virtuale(comando_utente))
10
11  def assistente_virtuale(comando):
12      if comando == "Qual è la data di oggi?":
13          oggi = datetime.datetoday()
14          risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15      elif comando == "Che ore sono?":
16          ora_attuale = datetime.datetime.now().time()
17          risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18      elif comando == "Come ti chiami?":
19          risposta = "Mi chiamo Assistente Virtuale"
20      else:
21          risposta = "Non ho capito la tua domanda."
22      return risposta
```

BREAKDOWN PRIMA SEZIONE CODICE (R. 3-9)

- Inizia con loop infinito “While True” (r.3) senza doppi punti successivi, successivamente la condizione if else utilizza l’indentazione per “break” in maniera errata (r.7) quindi entrambi non verranno svolti correttamente rompendo il ciclo.

```
while True
    comando_utente = input("Cosa vuoi sapere? ")
    if comando_utente == "esci":
        print("Arrivederci!")
    break
```

- Dopo “else:” l’output è completamente errato, ignorando che si richiama una funzione definita successivamente il che già non funzionerebbe, all’interno della chiamata alla funzione assistente_virtuale l’argomento assegnato “comando_utente” (r.9) non combacia con quello della funzione che invece dichiara l’argomento “comando” (r.11) rendendo impossibile controllo input utente per comando riconosciuto.

```
    if comando_utente == "esci":
        print("Arrivederci!")
    break
    else:
        print(assistente_virtuale(comando_utente))

def assistente_virtuale(comando):
    if comando == "Qual è la data di oggi?":
```

- La chiamata alla funzione assistente_virtuale (r.11) si trova prima ancora che questa funzione venga definita rendendo completamente l’operazione, in quanto il codice richiesto per ottenere la risposta in output non posso essere svolti.

```
def assistente_virtuale(comando):
    if comando == "Qual è la data di oggi?":
        oggi = datetime.datetoday()
        risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
    elif comando == "Che ore sono?":
        ora_attuale = datetime.datetime.now().time()
        risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
    elif comando == "Come ti chiami?":
        risposta = "Mi chiamo Assistente Virtuale"
    else:
        risposta = "Non ho capito la tua domanda."
    return risposta

while True
    comando_utente = input("Cosa vuoi sapere? ")
    if comando_utente == "esci":
        print("Arrivederci!")
    break
    else:
        print(assistente_virtuale(comando_utente))
```

ignorando altri errori, l'ordine funzione/chiamata sarebbe dovuta essere questa

BREAKDOWN SECONDA SEZIONE CODICE (R. 11-22)

- Definisce una funzione ma l'argomento non coincide con quello precedentemente utilizzato e viceversa per la chiamata come già determinato.
- Utilizzo di comando "if else = = " (r.12-15-18-21) in questo modo non è ottimale in quanto non prende in considerazione errori di scrittura comandi. Sarebbe opportuno creare una lista di variabili string (scritte in lowercase) Es. comandi_assistente [X, Y, Z], metterla in output su schermo così che utente sia consapevole di opzioni, infine chiedere un singolo comando=str(input("Comando Utente")).lower (per normalizzare tutti caratteri alpha in lowercase) così che con un ciclo di controllo if comando not in comandi_assistente tutti i comandi non riconosciuti possano dare output messaggio di errore e uscire dal ciclo. Il comando "exit" invece può essere una semplice condizione if come le altre. Ecco un esempio:

```
comandi_assistente=["data", "ora", "introduzione", "exit"]

def assistente_virtuale(comando):

    if comando not in comandi_assistente:
        print("Errore, prova di nuovo")

    elif comando == "data":
        oggi=datetime.date.today()
```

- La funzione datetime.datetoday() (r.13) è un errore di sintassi di datetime.date.today() come visto da documentazione python, funzione che restituisce un oggetto con data corrente.

```
class datetime.date
    An idealized naive date, assuming the current Gregorian calendar always was, and always will be, in effect.
    Attributes: year, month, and day.

    classmethod date.today()
        Return the current local date.
```

- Risposta è configurata male

```
risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
```

Sarebbe più facile fare così senza bisogno di assegnazioni:

```
print(f"La data di oggi è: {oggi.strftime('%d/%m/%Y')}")
```

.strftime(formato) (r.14) è almeno utilizzato correttamente come indicato da documentazione.

```
date.strftime(format)
    Return a string representing the date, controlled by an explicit format string.
```

```
.strftime('%a %d %b %Y, %I:%M%p')
```

- La funzione datetime.datetime.now().time() (r.16) estrae, grazie a .time() l'ora corrente in maniera corretta, ma in sé l'assegnazione di un valore alla variabile "risposta" (r.17) è formulata male nello stesso modo di quella precedente, sarebbe più semplice restituire un risultato direttamente con print.

Come avrei fatto io codice:

Su notepad per non aprire visual studio, mantiene più o meno la stessa architettura ma risolve ordine, indentazioni e errori di sintassi dove sono stati trovati (spero). Rimuove il bisogno di assegnare una variabile "risposta", Utilizza una libreria dove posso inserire comandi riconosciuti e fa un controllo iniziale di errori compilazione comandi. Normalizza input utente in lowercase nel caso sbagli.

```
import datetime

comandi_assistente=["data", "ora", "introduzione", "exit"]

def assistente_virtuale(comando):

    if comando not in comandi_assistente:
         print("Errore, prova di nuovo")

    elif comando == "data":
         oggi=datetime.date.today()
         print(f"La data di oggi è: {oggi.strftime('%d/%m/%Y')}")
    elif comando == "ora":
         ora_attuale=datetime.datetime.now().time() #.now() prende date e ora corrente .time() prende solo ora
         print(f"L'ora corrente è: {ora_attuale.strftime('%H:%M')}")
    elif comando == "introduzione":
         print("Mi chiamo Assistente Virtuale")
    else:
         return

while True:
  com=str(input("Inserisci un comando tra quelli disponibili, exit per uscire.\nComando:")).lower
  if com == "exit":
      print("Arrivederci!")
      break
  print(f"Lista Comandi:\n{comandi_assistente}")
  assistente_virtuale(com)
```